

SERVER CARD

CLOUD-01

CONTAINERIZED MICROSERVICES

TYPE: Cloud-Native Architecture

COST: 4 Budget (container orchestration platform)

COMPLEXITY: 3/4 (requires Kubernetes expertise)

SUPPORTED PLATFORMS: AWS ECS, Azure Container Instances, Google Cloud Run, Kubernetes

APPLICATION TYPE: Modern cloud-native applications built as multiple small services

DESCRIPTION

Applications broken into microservices running in containers (Docker). Each service is a separate container that can scale independently. Containers are orchestrated by a platform (Kubernetes, Docker Swarm, ECS).

KEY CHARACTERISTICS

Services: Application broken into small, focused services; Containers: Each service runs in a container (isolated environment); Orchestration: Platform automatically manages container deployment, scaling, and recovery; Scalability: Each service can scale independently based on demand; DevOps: Infrastructure and code are tightly integrated (Infrastructure as Code); Cloud-Native: Built for cloud (ephemeral resources, auto-scaling, auto-healing);

KEY ADVANTAGES

Scalability: Auto-scale individual services based on load; Resilience: Container failure doesn't affect entire application; Efficiency: Containers share OS kernel (more efficient than VMs); Velocity: Deploy microservices independently (faster development); Cost: Pay only for resources used (serverless principles);

KEY CONCERNS (SECURITY PERSPECTIVE)

Complexity: Many moving parts, many potential attack surfaces; Container Escape: Attacker in one container might escape to host; Supply Chain: Container images come from registries (may be compromised); Secrets Management: Container needs credentials/API keys (risk of exposure); Lateral Movement: Multiple containers in same cluster; Network policies must isolate; Compliance: Tracing data across microservices complicated; Logging: Distributed across many containers (aggregation required);

CONTINUED IN MODULE RULES

INCIDENT ZERO NETWORK BUILDING

SERVER CARD

CLOUD-02

SERVERLESS/FUNCTION-AS-A-SERVICE

TYPE: Cloud-Native Compute (Abstracted Infrastructure)

COST: 3 Budget (minimal infrastructure management)

COMPLEXITY: 2/4 (simple from ops perspective, but different security model)

SUPPORTED PLATFORMS: AWS Lambda, Azure Functions, Google Cloud Functions, IBM Cloud Functions

USE CASES: Event-driven processing, API backends, scheduled jobs, data transformation

DESCRIPTION

Functions deployed to serverless platform. You write code, platform handles scaling, infrastructure, and execution. You pay only for actual execution time (milliseconds).

KEY CHARACTERISTICS

No Infrastructure: You don't manage servers/containers/VMs; Event-Driven: Functions triggered by events (API call, message queue, schedule, etc.); Auto-scaling: Automatically scales from zero to thousands of concurrent executions; Stateless: Functions should not maintain state (state in database/message queue); Time-Limited: Function has timeout (usually 15 minutes max); Cost Model: Pay per execution/millisecond (very cheap if infrequently called);

KEY ADVANTAGES

Simplicity: Write code, deploy, done (no servers to manage); Cost: Pay only for execution time (not for idle servers); Scalability: Automatic scaling without configuration; Velocity: Rapid deployment cycle (seconds); Reduced Attack Surface: No server to compromise (provider manages infrastructure);

KEY CONCERNS (SECURITY PERSPECTIVE)

Dependency Management: Function depends on libraries (vulnerable packages); Secrets in Environment Variables: Function needs credentials (hardcoded or env vars); Supply Chain: Libraries may be compromised (dependency attack); Monitoring Blind Spot: Functions are ephemeral (logs must be aggregated); Cold Start Attacks: Function startup time can leak information; Privilege Escalation: Function may have overly broad IAM permissions; Denial of Service: Function can be invoked repeatedly (cost/resource attack); Compliance: Difficult to audit function execution (multi-tenant platform);

CONTINUED IN MODULE RULES

INCIDENT ZERO NETWORK BUILDING

SERVER CARD

CLOUD-03

DATABASE-AS-A-SERVICE (MANAGED DATABASE)

TYPE: Cloud-Managed Data Layer

COST: 5 Budget (pay per GB + IO operations)

COMPLEXITY: 1/4 (cloud provider manages infrastructure)

SUPPORTED PLATFORMS: AWS RDS, Azure SQL Database, Google Cloud SQL, MongoDB Atlas, DynamoDB

DATABASE TYPES: Relational (SQL Server, PostgreSQL, MySQL, Oracle), NoSQL (MongoDB, DynamoDB, Cassandra)

DESCRIPTION

Database managed entirely by cloud provider. You provision database capacity, cloud provider handles: backups, patching, failover, replication, encryption, monitoring.

KEY CHARACTERISTICS

Managed: Cloud provider manages infrastructure (patches, backups, upgrades); Automated Backup: Point-in-time recovery (no manual backup configuration); Replication: Automatic geo-replication for disaster recovery; Encryption: Encryption at rest and in transit (built-in); Scaling: Can scale storage and compute without downtime; Monitoring: Built-in monitoring and alerting; Compliance: Provider maintains compliance certifications (SOC 2, HIPAA, PCI-DSS);

KEY ADVANTAGES

Reliability: Provider manages availability (99.99% SLA typical); Security: Provider manages patching and security updates; Compliance: Provider handles compliance certifications; Cost: No ops team needed to maintain database; Disaster Recovery: Automated backups and geo-replication; Scalability: Transparent scaling without downtime;

KEY CONCERNS (SECURITY PERSPECTIVE)

IAM Configuration: Database access via IAM roles (misconfiguration exposes data); Network Exposure: Database may be internet-accessible (must restrict); Encryption Keys: Cloud provider manages keys (limited control); Audit Logging: Must enable audit logging (not always on by default); Data Residency: Where is data stored geographically; Backup Security: Backups must be encrypted and access-controlled; Supply Chain: Vendor is now part of attack surface; Multi-Tenant: Data may share hardware with other customers (trust model);

CONTINUED IN MODULE RULES

INCIDENT ZERO NETWORK BUILDING

SERVER CARD

CLOUD-04

CONTENT DELIVERY NETWORK (CDN)

TYPE: Edge Computing / Performance Enhancement

COST: 4 Budget (pay per GB transferred + requests)

COMPLEXITY: 2/4 (configuration required, but well-documented)

SUPPORTED PLATFORMS: CloudFlare, AWS CloudFront, Azure CDN, Google Cloud CDN, Akamai

USE CASES: Static assets (images, JS, CSS), web application acceleration, DDoS mitigation

DESCRIPTION

CDN caches content across globally distributed edge servers. When users request content, they get it from nearest edge server (fast). Reduces load on origin server and improves user experience.

KEY CHARACTERISTICS

Distributed Caching: Content cached at 100+ edge locations worldwide; Fast Delivery: Users get content from nearest edge (milliseconds); Origin Protection: Origin server behind CDN (hides IP, reduces load); DDoS Protection: CDN absorbs DDoS attacks before reaching origin; SSL/TLS Termination: CDN handles encryption (origin can use HTTP); Rate Limiting: Can rate-limit requests per IP; Security Headers: Can inject security headers (HSTS, CSP, etc.);

KEY ADVANTAGES

Performance: Content delivery 100x faster globally; DDoS Protection: Built-in DDoS mitigation; Security: Origin IP hidden, security scanning at edge; Cost Efficiency: Reduces origin server bandwidth costs; Geo-location: Can route users to different content based on location;

KEY CONCERNS (SECURITY PERSPECTIVE)

Cache Poisoning: Attacker poisons CDN cache with malicious content; Origin Bypass: Attacker finds origin IP, bypasses CDN security; SSL Stripping: CDN can decrypt traffic (must trust CDN provider); Cookie Security: Sensitive cookies may be cached (GDPR/privacy issue); Personalization: Cached content loses personalization (may expose user data); Configuration Mistakes: Wrong cache settings may cause sensitive content; Origin Protection: Still need to protect origin server (CDN is not complete solution); Bot Attack: Attackers can still target origin through CDN;

CONTINUED IN MODULE RULES

INCIDENT ZERO NETWORK BUILDING

SERVER CARD

LEGACY-01

MAINFRAME SYSTEM

TYPE: Legacy Business-Critical Infrastructure

COST: 15 Budget (expensive to operate)

COMPLEXITY: 4/4 (extremely complex, specialized expertise required)

AVAILABILITY REQUIREMENT: 99.99% (financial/mission-critical)

SUPPORTED OPERATING SYSTEM: Mainframe OS (z/OS, VSE, VM), Not patchable

DESCRIPTION

Large centralized mainframe computer running business-critical applications. Banks, insurance companies, government agencies, and utilities often depend on mainframes for core operations. Mainframes are designed for availability and stability, but security model is outdated (predates internet security concerns).

KEY CHARACTERISTICS

Age: Often 20-30+ years old; Software: Custom applications written in COBOL or other legacy languages; Security: Perimeter security model (assume all internal traffic is trusted); Expertise: Few experts remain (knowledge leaving the industry); Vendor Support: Vendor may no longer exist; in-house expertise critical; Cost of Replacement: \$5M-100M+ (impossible to replace);

KEY CONCERNS

No regular security patches available (vendor EOL); Known CVEs published but cannot be patched; Legacy protocols (unencrypted connections, weak authentication); Access control systems predate modern security standards; Directly connected to corporate network (not isolated); often contains entire organization's critical data; Business process depends on it (replacement would take years);

DEFENSE STRATEGY (CANNOT FIX, MOST ISOLATE)

Network Isolation: Mainframe in restricted segment, minimal connectivity; Firewall Rules: Only authorized systems can connect to specific ports; Monitoring: SIEM must monitor all mainframe connections (detect anomalies); Assumption of Breach: Assume mainframe will be compromised; defend everything else; No Direct User Access: Users should not directly access mainframe (access via secure application tier); Log Aggregation: All mainframe activities logged centrally (immutable audit trail);

NETWORK INTERACTIONS

Requires: Firewall rules isolating mainframe; Requires: Network Segmentation to restrict access; Requires: SIEM monitoring for anomaly detection; Incompatible with: Direct internet access (perimeter firewall only); Incompatible with: Cloud integration/data residency and connectivity issues);

CONTINUED IN MODULE RULES

INCIDENT ZERO NETWORK BUILDING

SERVER CARD

LEGACY-02

CUSTOM BUSINESS APPLICATION

TYPE: Legacy Business-Critical Software

COST: 8 Budget (application licensing/support)

COMPLEXITY: 3/4 (difficult to update or replace)

AVAILABILITY REQUIREMENT: 99% (business depends on it)

OPERATING SYSTEM: Windows Server 2008/2012, or older Linux, or proprietary OS

DESCRIPTION

Specialized business application written by external vendor or in-house team that is no longer maintained. Application may be: Accounting system (custom financial software); Manufacturing control system (production scheduling); Insurance claims system; Legal case management system; ERP (Enterprise Resource Planning) system; The application is mission-critical but: Vendor no longer provides updates (company went out of business, or stopped supporting); Cost to replace: \$50M-\$5M+ (economically infeasible); Business process deeply embedded in application (replacement would require major restructuring); Knowledge of application is scarce (original developers left, documentation poor);

KEY CHARACTERISTICS

Age: Often 10-20+ years old; Language: Written in outdated languages (Visual Basic 6, old Java, COBOL); Vendor: Vendor may no longer exist; purchased "as-is"; Customization: Heavily customized for organization (not standard product); Expertise: Few people understand the code; Database: Proprietary or very old database (SQL Server 2005, Oracle 10g, etc.); Integration: Deeply integrated into business workflow;

KEY CONCERNS

Application may contain hardcoded credentials; Application may store passwords in plaintext; Application may use unencrypted connections to database; Application may have SQL injection vulnerabilities (pre-2005 development); Application may lack audit logging; Upgrading operating system may break application; Patches to operating system may be incompatible with application; Security testing tools may break application;

DEFENSE STRATEGY (ISOLATE AND MONITOR)

Application Server Isolation: Application in restricted segment; Database Isolation: Application database encrypted and access-controlled; Network Firewall: Only users/systems needing application can access it; No Direct Internet Access: Application never exposed to internet; SIEM Monitoring: All application access logged and monitored; Assumption: Assume application has SQL injection or similar vulnerabilities; defend database; Access Control: Limit who can use application (risk vs. benefit analysis);

CONTINUED IN MODULE RULES

INCIDENT ZERO NETWORK BUILDING

SERVER CARD

LEGACY-03

INDUSTRIAL CONTROL SYSTEM (ICS)

TYPE: Operational Technology (OT) / Safety-Critical

COST: 12 Budget (specialized equipment)

COMPLEXITY: 4/4 (extremely specialized, safety implications)

AVAILABILITY REQUIREMENT: 99.99+ (safety-critical, cannot fail)

OPERATING SYSTEM: Proprietary Industrial OS, real-time OS, or very old Linux

DESCRIPTION

Specialized system that controls physical machinery, manufacturing processes, power generation, utilities, or other critical infrastructure. Examples: Manufacturing: Assembly line control, robotic systems; Utilities: Power distribution, water treatment, smart grid; Building Systems: HVAC, access control, fire systems; Transportation: Traffic signals, rail systems; Industrial Control Systems (ICS) are purpose-built for reliability and real-time control, not information security. They predate cybersecurity concerns.

KEY CHARACTERISTICS

Purpose: Controls physical machinery or critical infrastructure; Real-time: Must respond in milliseconds (cannot tolerate latency); Reliability: Designed for 99.999+ availability; Security: Predates modern security (no authentication, no encryption); Isolation: Historically air-gapped (not connected to corporate network); Expertise: Requires specialized engineering expertise (electrical, mechanical, control systems); Cost of Downtime: \$10K-\$3M+ per minute of downtime; Lifecycle: 10-30 year lifespan (different than IT systems);

KEY CONCERNS (FROM IT PERSPECTIVE)

Cannot be patched (real-time systems cannot tolerate OS updates); Cannot tolerate intrusion detection (latency would affect operations); Cannot install antivirus/updates (would impact real-time responses); Cannot use modern encryption (adds latency); Uses legacy protocols (Modbus, Profibus, DNP3 - no security built-in); Often air-gapped; now being connected to corporate network (exposes vulnerability); If compromised, physical safety implications (machinery malfunction, power outage, etc.);

KEY CONCERNS (FROM ICS PERSPECTIVE)

Availability is paramount (security is secondary); Adding security controls might affect uptime; ICS engineers don't understand IT security; IT security tools and practices may break ICS; ICS changes must be tested extensively (downtime is unacceptable);

CONTINUED IN MODULE RULES

INCIDENT ZERO NETWORK BUILDING

SERVER CARD

LEGACY-04

OBSOLETE OPERATING SYSTEM

TYPE: Legacy Infrastructure Running Unsupported OS

COST: 5 Budget (cheap hardware, but difficult to support)

COMPLEXITY: 3/4 (difficult to manage with modern security tools)

AVAILABILITY REQUIREMENT: 80-95% (business can tolerate occasional downtime)

OPERATING SYSTEM EXAMPLES: Windows XP, Windows Server 2003, old Linux kernels, custom UNIX variants

DESCRIPTION

Systems running operating systems that are no longer supported by vendor or open-source community. Vendor has stopped releasing security patches. Public exploit code for known vulnerabilities exists and is freely available.

EXAMPLES

Windows XP (support ended 2014); Windows Server 2003 (support ended 2015); Linux 2.4/2.6 kernels (EOL for 10+ years); Solaris 8/9 (Sun Microsystems EOL); Other commercial UNIX variants EOL decades ago;

KEY CHARACTERISTICS

Support: No security patches from vendor; Exploits: All known vulnerabilities are published (no zero-days); Tools: Modern security tools often don't run on EOL OS; Compatibility: Cannot run modern applications; Patching OS: OS upgrade would break dependent applications; Cost of Replacement: Application replacement cost makes OS upgrade prohibitive;

KEY CONCERNS

All known vulnerabilities can be exploited (no patches); Public exploits readily available (trivial for attacker); Modern malware often targets these systems (profitable attacks); Ransomware frequently targets EOL systems (known vulnerabilities); System cannot run modern antivirus/EDR (not compatible); System cannot use modern encryption protocols; System is prone to exploitation for lateral movement into modern systems;

DEFENSE STRATEGY (ASSUME COMPROMISE, ISOLATE)

Assumption: Assume EOL system will be compromised (all vulnerabilities are public); Network Isolation: EOL system in restricted segment, minimal connectivity; Firewall Rules: Only necessary traffic to/from EOL system; No Lateral Movement: Firewall rules prevent movement from EOL system to others; No Sensitive Data: Do not store sensitive data on EOL system; Monitoring: SIEM monitors EOL system closely (detect compromise signs); Replacement Planning: Have timeline to replace/upgrade EOL system; Physical Security: Restrict physical access to EOL systems;

CONTINUED IN MODULE RULES

INCIDENT ZERO NETWORK BUILDING

